ROS 2 Autonomous Mobile Robot

Engineering report: architecture, code walkthrough and how to rebuild the whole system yourself

Project	BE project – differential-drive robot with 2D lidar mapping, PC remote control and autonomous exploration
Hardware	Raspberry Pi 5 · Arduino Mega 2560 · H-bridge + 4 DC motors (no encoders) · YDLIDAR X2
Software	ROS 2 Jazzy (Pi and PC container) · slam_toolbox · Nav2 · Python 3 · C++ lidar driver · Arduino C++
Version	v2 (robot_v2.zip, workspace robot_ws)
Author	Melek Chourabi – ENIT, génie électrique
Date	5 November 2026

Key idea. This report explains **why** each part exists, **how** the data flows from the lidar to the map and from your keyboard to the wheels, and **what every important line of code does**. Chapter 15 is a step-by-step plan to rebuild the project from an empty folder, so you can own it rather than just run it. All numbers come from the real code and configuration files of the project.

Warning. Honest status: the software was verified offline (unit tests, simulation, a real browser test of the panel). It has not been validated end-to-end on your physical robot yet. Chapter 16 lists exactly what is tested and what is not.

Contents

Contents 2

1. Goals and design principles 4

1.1 Design principles used in the code . . . 4

1.2 Vocabulary you need . . . 4

2. Hardware and physical architecture 5

2.1 Rules that avoid hardware problems . . . 5

2.2 Robot geometry used by the software . . . 5

3. System architecture 5

3.1 Layers . . . 6

3.2 The nodes . . . 7

3.3 Topics and QoS . . . 7

3.4 Coordinate frames (TF) . . . 7

4. End-to-end data flows 8

4.1 Control path: from a key press to the wheels . . . 8

4.2 Sensing path: from the lidar to the map in your browser . . . 9

4.3 Worst-case stopping distance (engineering check) . . . 9

5. Firmware: mega_motor_driver.ino 9

5.1 Driving one side . . . 9

5.2 Parsing and the watchdog . . . 10

6. The base: kinematics.py, motors.py, base_node.py 10

6.1 Differential-drive mathematics (kinematics.py) . . . 10

6.2 Talking to the Mega (motors.py) . . . 11

6.3 The base node (base_node.py) . . . 11

7. Perception: lidar driver, TF and scan filter 12

7.1 The YDLIDAR driver . . . 12

7.2 scan_filter.py . . . 12

8. Command arbitration and safety: cmd_mux 12

8.1 Modes . . . 12

8.2 The step() function, line by line . . . 13

8.3 Clearance from the lidar scan . . . 13

8.4 The ROS wrapper (cmd_mux.py) . . . 13

9. Autonomous exploration: explore.py 13

10. Mapping and navigation configuration 14

10.1 slam_toolbox (config/slam_v2.yaml) . . . 14

10.2 Nav2 (optional, NAV=true) . . . 15

10.3 Health and autosave . . . 15

11. The PC web panel **15**

11.1 Security model . . . 15

11.2 How the map reaches the browser . . . 16

12. Launch system and configuration files **16**

12.1 The important remappings . . . 16

12.2 Where to change what . . . 17

13. Networking: Pi ↔ PC, Docker, DDS **17**

14. Operation, calibration and tuning **17**

14.1 Daily procedure . . . 17

14.2 Calibration sequence . . . 17

15. How to rebuild the project yourself **18**

15.1 Minimal skeleton: a ROS 2 node in Python . . . 18

15.2 Package layout . . . 19

16. Testing strategy **19**

17. Troubleshooting map **20**

18. Limitations and roadmap **20**

18.1 What this system is not . . . 20

18.2 Recommended upgrades, in order . . . 20

Appendix A – Command cheat sheet **21**

Appendix B – Glossary of the code names **21**

1. Goals and design principles

The robot has three jobs: **(1)** be driven from a PC while you watch a live map and the lidar, **(2)** build a map of a room with SLAM, and **(3)** move on its own, either by simple reactive exploration or by Nav2 navigation to a point clicked on the map. Everything is designed around a small, cheap robot with **no wheel encoders**, which is the main constraint on map quality.

1.1 Design principles used in the code

- **Separate thinking from driving.** The Pi runs all the intelligence (ROS 2). The Arduino Mega only turns "left PWM, right PWM" into pin states. If the Pi crashes, the Mega alone can only stop (watchdog).
- **One writer to the motors.** Many nodes want to move the robot (panel, explorer, Nav2). Only one is listened to at a time (the *mode*), and everything passes through one safety node.
- **Fail safe.** Silence means stop: no command for 0.5 s, no lidar for 1 s, no serial command for 0.4 s, e-stop, or a lost USB cable all end with zero motor output.
- **Pure logic separated from ROS.** The decisions (mux.py, kinematics.py, scan_filter.filter_ranges, the Wanderer) are plain Python functions/classes. That is why they could be unit-tested without a robot.
- **Everything configurable in YAML** (base.yaml, robot.yaml, slam_v2.yaml) and one launch file starts the whole robot.

1.2 Vocabulary you need

Term	Meaning in this project
Node	A running program that does one job (e.g. cmd_mux). Nodes talk only through topics.
Topic	A named stream of messages with a type, e.g. /scan carries LaserScan. Publishers write, subscribers read.
QoS	Delivery rules of a topic. Sensors use Best Effort (drop old data); /map uses Reliable + Transient Local (a late subscriber still receives the last map).
TF	The tree of coordinate frames (map → odom → base_link → lidar_link). It tells every node where things are.
Odometry	Estimate of how far the robot moved. Here dead-reckoned from the motor commands (no encoders).
SLAM	Simultaneous Localisation And Mapping: build the map and find the robot pose in it, from lidar scans.
Lifecycle node	A node with states (unconfigured → inactive → active). slam_toolbox must be activated or it publishes no map.
DDS domain	ROS 2 machines only see each other if they share ROS_DOMAIN_ID (42 here) and the network allows multicast.
PWM / duty	Pulse-width modulation: motor power as a fraction (−1…+1 in Python, −255…255 on the Mega).

2. Hardware and physical architecture

Component	Role	Interface / key facts
Raspberry Pi 5	Runs ROS 2, SLAM, Nav2, drivers	Ubuntu 24.04, Jazzy. Needs a real 5 V / 5 A supply: undervoltage caused the USB over-current errors you saw.
YDLIDAR X2	2D scan, ~8 m range, ~10 Hz	USB (CP2102) → /dev/ttyUSB*, 115200 baud, single-channel; motor is controlled through DTR. Used on the data USB only.
Arduino Mega 2560	Real-time motor interface	USB → /dev/ttyACM*, 115200 baud, text protocol. Resets when the port opens (2 s boot wait).
H-bridge (L298N style)	Power stage	DRIVER_TYPE 1: per side IN1, IN2 (direction) + EN (PWM). Default pins L: 22, 23, 5; R: 24, 25, 6 (you updated yours – edit the #defines).
DC motors ×4	Two per side, wired in parallel	Skid steer: left pair on one bridge channel, right pair on the other.
PC	Operator console	Ubuntu with ROS Lyrical; ROS-Jazzy Docker container for the panel; links to the Pi by RJ45 (Pi = 192.168.50.2).

2.1 Rules that avoid hardware problems

- **Common ground** between the Mega and the H-bridge (and the motor battery negative). Without it the PWM signals have no reference.
- Remove the ENA/ENB jumpers on an L298N so that the PWM pins really control the speed.
- Motors on their own battery; never power them from the Pi. Add a physical switch on the motor battery: the software e-stop is not a certified safety function.
- Use */dev/serial/by-id/...* names (robot.env) because ttyUSB0/ttyACM0 numbers change with plug order.

2.2 Robot geometry used by the software

The lidar is not at the centre of the robot: it sits 0.068 m behind *base_link* (parameter lidar_x = −0.068) at height 0.12 m. The safety code needs the distance from the **body** to an obstacle, not from the lidar, so it subtracts the offsets.

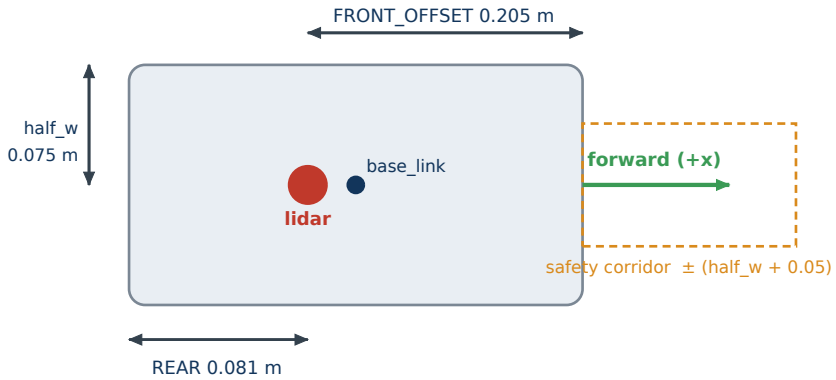


Figure 1 – Footprint seen from the lidar. FRONT_OFFSET, REAR_OFFSET and half_width come from robot.yaml (measure your robot and edit them).

3. System architecture

Two computers cooperate over one ROS 2 network. The Pi owns the robot; the PC is a console. The PC panel runs inside a Docker container with ROS 2 **Jazzy** because the Pi runs Jazzy while your PC has Lyrical: two different ROS distributions do not reliably exchange data, so the container makes both ends speak the same dialect.

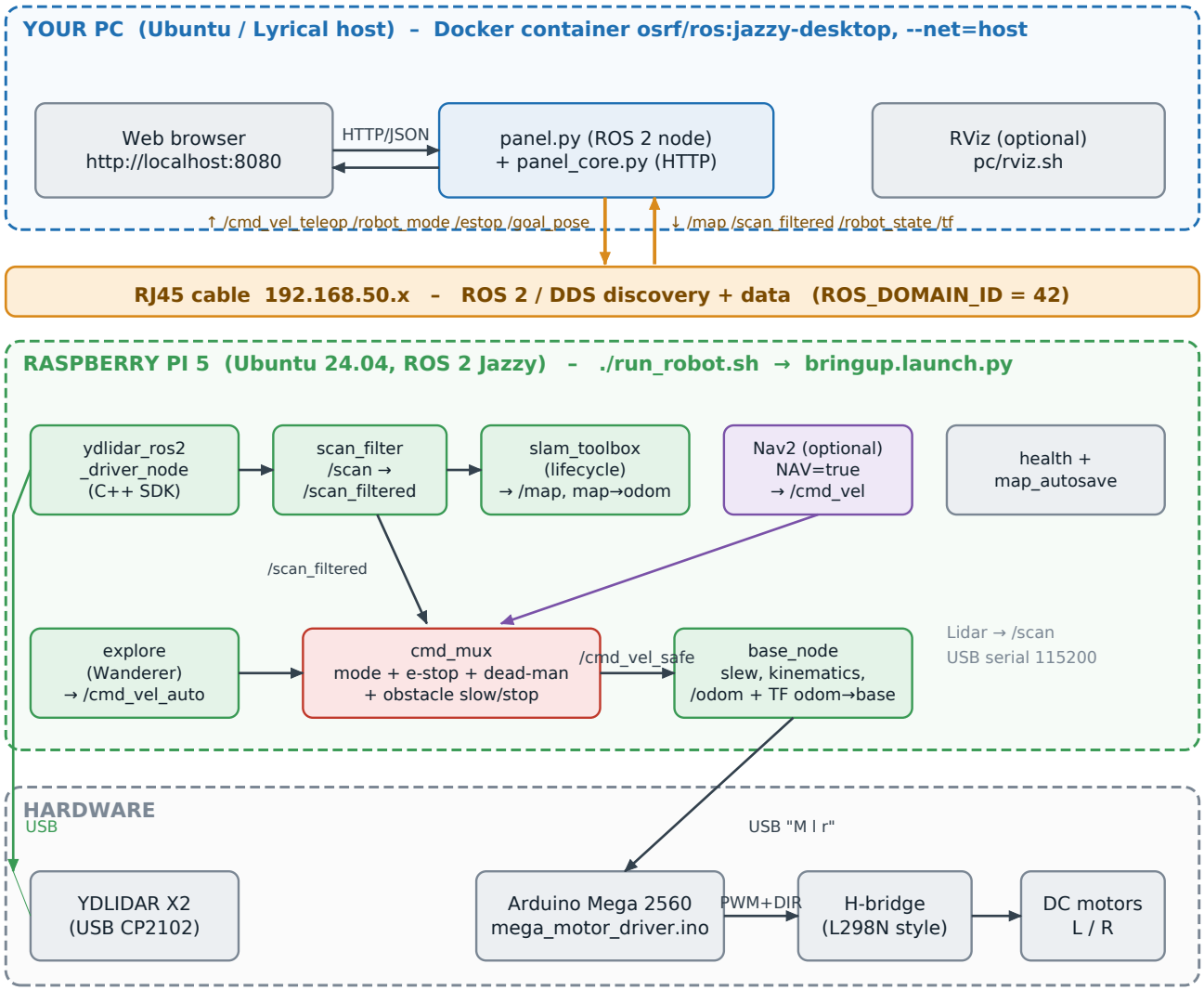


Figure 2 – Physical and software architecture. Green: nodes that produce data or motion; red: safety layer; purple: optional Nav2.

3.1 Layers

Layer	Contents	Why
Operator (PC)	Browser UI, panel.py, optional RViz	Visualise, send intent (mode, key presses, goal). Never talks to the motors directly.
Autonomy (Pi)	slam_toolbox, Nav2, explore.py	Decide *what* to do: map, plan, wander.
Safety (Pi)	scan_filter, cmd_mux, health	Decide what is *allowed*: filter sensors, limit commands, watch for failures.
Base (Pi)	base_node, motors.py, kinematics.py	Turn velocity commands into wheel PWM and odometry.
Firmware (Mega)	mega_motor_driver.ino	Drive pins, 400 ms watchdog, (optional) read encoders.
Power / actuators	H-bridge, motors	Physical motion.

3.2 The nodes

Node (executable)	File	Subscribes	Publishes
ydlidar_ros2_driver_node	ydlidar_ros2_driver (C++)	–	/scan
static_tf_lidar	tf2_ros (in lidar.launch.py)	–	TF base_link → lidar_link
scan_filter	scan_filter.py	/scan	/scan_filtered
slam_toolbox	slam_toolbox (online_async)	/scan_filtered, TF odom→base_link	/map, TF map→odom
explore	explore.py	/scan_filtered	/cmd_vel_auto
cmd_mux	cmd_mux.py + mux.py	/cmd_vel_teleop, /cmd_vel_auto, /cmd_vel, /robot_mode, /estop, /scan_filtered	/cmd_vel_safe, /robot_state
base_node	base_node.py	/cmd_vel_safe	/odom, TF odom→base_link, /base_status
health	health.py	/scan, /odom, /map, /base_status	/robot_health
map_autosave	map_autosave.py	/map	files in ~/maps/autosave
Nav2 stack (optional)	nav2_bringup	/map, /scan, TF, /goal_pose	/cmd_vel
robot_panel (PC)	panel.py	/map, /scan_filtered, /robot_state, /robot_health, TF	/cmd_vel_teleop, /robot_mode, /estop, /goal_pose

3.3 Topics and QoS

Topic	Type	From → To	Notes
/scan	sensor_msgs/LaserScan	driver → filter, health	Best Effort (sensor data)
/scan_filtered	LaserScan	filter → SLAM, mux, explorer, panel	Same geometry, bad beams = inf
/cmd_vel_teleop	geometry_msgs/Twist	panel → mux	Used in mode manual
/cmd_vel_auto	Twist	explorer → mux	Used in mode explore (remapped from /cmd_vel)
/cmd_vel	Twist	Nav2 → mux	Used in mode nav
/cmd_vel_safe	Twist	mux → base_node	The only command that reaches the motors
/odom	nav_msgs/Odometry	base_node → health	Pose + twist with covariance
/map	nav_msgs/OccupancyGrid	SLAM → panel, autosave, Nav2	Reliable + Transient Local; −1 unknown, 0 free, 100 occupied
/robot_mode	std_msgs/String	panel → mux	stop | manual | explore | nav
/estop	std_msgs/Bool	panel → mux	True latches the e-stop
/robot_state	String (JSON)	mux → panel	mode, estop, front/rear clearance, reason, blind
/robot_health	String (JSON)	health → panel	level ok/warn/error + problems + Pi stats
/base_status	String (JSON)	base_node → health	backend, Mega connected, odometry type, v, w
/goal_pose	PoseStamped	panel → Nav2	Click-and-drag on the map

3.4 Coordinate frames (TF)

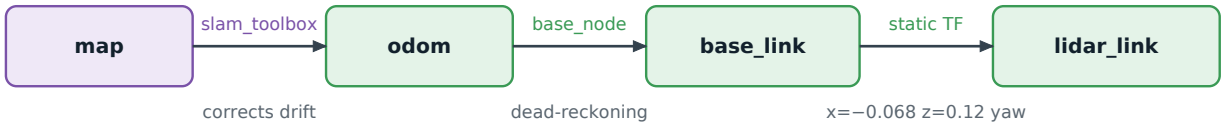


Figure 3 – TF tree. The panel draws the robot with lookup(map → base_link) and the lidar points with lookup(map → lidar_link).

Odometry alone drifts, so *odom → base_link* slowly becomes wrong. SLAM compares each scan with the map and publishes *map → odom* as a correction. The robot pose in the map is the product of the two transforms. This is why the map stays consistent even with poor odometry, and why it is important that the lidar is stable and the motors do not slip too much.

4. End-to-end data flows

4.1 Control path: from a key press to the wheels

#	Step	Where	Time scale
1	You hold ↑ in the browser; the page POSTs /api/cmd {v, w} repeatedly with a secret token	Browser → panel_core.py	repeated while held
2	send_cmd clamps (|v| ≤ 0.25, |w| ≤ 1.5) and publishes Twist on /cmd_vel_teleop	panel.py → DDS → Pi	ms
3	cmd_mux stores (v, w, time) for source "manual"	cmd_mux.on_cmd	ms
4	Every 50 ms (20 Hz) mux.step() applies the rules: mode, dead-man, blind-lidar, obstacle scaling, clamps (0.20 m/s, 1.2 rad/s)	cmd_mux.tick	50 ms
5	Result is published on /cmd_vel_safe	cmd_mux → base_node	20 Hz
6	base_node (50 Hz loop) applies acceleration limits, converts to wheel speeds, then to duty, then calls motors.set()	base_node.tick	20 ms
7	SerialMotors writes "M l r\n" (re-sent at least every 0.1 s)	motors.py → USB	≤ 100 ms
8	The Mega parses the line, sets IN1/IN2/EN pins and arms its 400 ms watchdog	firmware	µs
9	You release the key: step 1 stops; after 0.5 s the source is stale, so mux outputs zero; base_node ramps down	safety chain	≤ 0.5 s + ramp

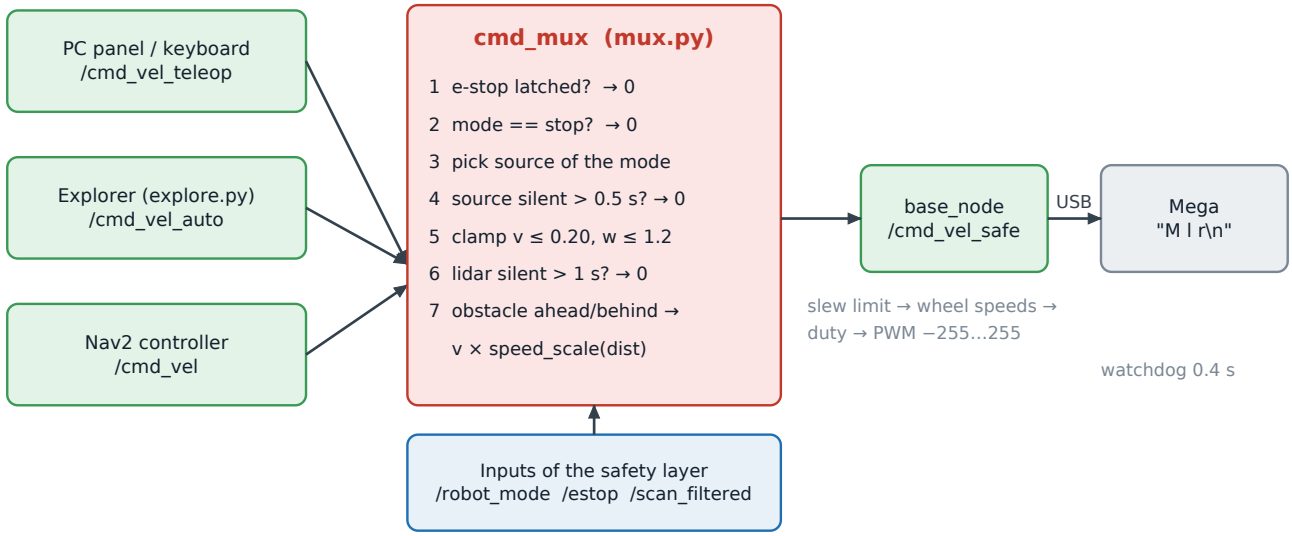


Figure 4 – Command chain. Three possible sources, one safety node, one path to the hardware.

4.2 Sensing path: from the lidar to the map in your browser

- ydlidar driver publishes */scan* (~10 Hz, up to ~8 m).
- scan_filter replaces invalid and isolated beams by inf and republishes */scan_filtered*.
- slam_toolbox consumes the scan and TF *odom* → *base_link*, matches the scan against its pose graph, publishes */map* (OccupancyGrid) and *map* → *odom*.
- panel.py converts the grid to bytes (−1 → 255), projects the scan points into the map frame, and exposes them as JSON through /api/state.
- index.html polls the state and draws the map, robot, trail and lidar points on a canvas.

4.3 Worst-case stopping distance (engineering check)

The software layers add delays. For the maximum speed v = 0.20 m/s: the dead-man needs up to 0.5 s to notice a silent source (0.20 × 0.5 = 0.10 m), then base_node ramps down with max_accel = 0.6 m/s² (0.20 / 0.6 = 0.33 s, extra distance ½·0.2·0.33 ≈ 0.033 m). Worst case is therefore about **13 cm** after the operator lets go. The obstacle layer reduces the speed linearly between 0.45 m and 0.10 m, so near a wall the robot is already slow. This is also why the e-stop is not instantaneous: the zero command is immediate, but the slew limiter still takes ≈ 0.33 s from full speed. A physical cut-off removes that delay.

5. Firmware: mega_motor_driver.ino

The Mega is deliberately dumb. It knows one thing: how to apply a signed PWM to each side. The protocol is plain text lines ending in a newline, which makes it testable from any serial terminal (and from arduino_test.py).

Direction	Line	Meaning
Pi → Mega	`M <left> <right>`	PWM −255…255 per side; the sign is the direction. Example: "M 120 -120" spins in place.
Pi → Mega	`S`	Stop both sides.
Pi → Mega	`P`	Ping; the Mega answers PONG.
Pi → Mega	`Z`	Reset the encoder counters (only if ENCODERS is 1).
Mega → Pi	`# mega_motor_driver ready`	Sent once after reset.
Mega → Pi	`T <ticksL> <ticksR>`	Encoder ticks every 50 ms (only if ENCODERS is 1).
Mega → Pi	`? <line>`	Echo of a line the Mega could not understand.

5.1 Driving one side

Listing 1 – H-bridge control (L298N-style: direction pins + one PWM)

```
static void driveSide(uint8_t in1, uint8_t in2, uint8_t pwm, uint8_t pwm2, int duty) {
#if DRIVER_TYPE == 1
  if (duty > 0)      { digitalWrite(in1, HIGH); digitalWrite(in2, LOW);  analogWrite(pwm,  duty); }
  else if (duty < 0) { digitalWrite(in1, LOW);  digitalWrite(in2, HIGH); analogWrite(pwm, -duty); }
  else               { digitalWrite(in1, LOW);  digitalWrite(in2, LOW);  analogWrite(pwm, 0); }
#else   // type 2: BTS7960 style, two PWM pins
  analogWrite(pwm,  duty > 0 ? duty : 0);
  analogWrite(pwm2, duty < 0 ? -duty : 0);
#endif
}
```

For an L298N, IN1/IN2 select the direction (HIGH/LOW = forward, LOW/HIGH = reverse, LOW/LOW = brake/coast) and the EN pin carries the PWM that sets the average voltage on the motor. *analogWrite* on a Mega PWM pin outputs ≈ 490 Hz, duty = value/255.

5.2 Parsing and the watchdog

Listing 2 – Command parsing and safety watchdog (simplified from the file)

```
case 'M': {
  char* p1; char* p2;
  long l = strtol(s + 1, &p1, 10);   if (p1 == s + 1) break;       // no number -> error
  long r = strtol(p1, &p2, 10);      if (p2 == p1) break;
  setMotors((int)l, (int)r);
  lastCmdMs = millis();               // feed the watchdog
  return;
}
...
// in loop():
if (!stopped && millis() - lastCmdMs > WATCHDOG_MS) setMotors(0, 0);   // 400 ms silence -> stop
```

- **Why the Mega resets on connection:** opening the USB serial port toggles DTR, which resets an Arduino. The Python side therefore waits *serial_boot_wait* = 2 s before talking.
- **Encoders:** with ENCODERS 1 the Mega counts quadrature pulses with interrupts on pins 2, 3, 18–21 and sends "T l r". The ROS side already supports it (use_encoders:=true). It is the single best upgrade for map quality.
- **Pin configuration:** everything you may need to change is in the CONFIG block (DRIVER_TYPE, L_IN1…R_PWM2, WATCHDOG_MS, ENCODERS).

6. The base: kinematics.py, motors.py, base_node.py

6.1 Differential-drive mathematics (kinematics.py)

A skid-steer robot is controlled by the speed of its left and right sides. With forward speed v (m/s), turning rate ω (rad/s, counter-clockwise positive) and wheel separation L = 0.1471 m:

Equations 1

```
v_left  = v - ω · L/2
v_right = v + ω · L/2          # wheel_speeds(v, w, separation)

v = (v_left + v_right) / 2
ω = (v_right - v_left) / L     # body_velocity(...) times calibration factors
```

Example: v = 0.10 m/s, ω = 0.5 rad/s gives v_left = 0.10 − 0.0368 = 0.0632 m/s and v_right = 0.1368 m/s. To turn in place (v = 0, ω = 1 rad/s) the sides run at ∓0.0736 m/s.

Speed → duty (the cause of "the motors move too slowly")

Listing 3 – Linear map that skips the stall zone

```
def speed_to_duty(speed, max_speed, min_duty):
    d = clamp(speed / max_speed, -1.0, 1.0)
    a = abs(d)
    if a < 0.02:
        return 0.0
    return math.copysign(min_duty + (1.0 - min_duty) * a, d)
```

DC motors do not move below some PWM (the stall threshold). The mapping therefore starts at **min_duty** (0.25 by default) for any non-zero command and rises linearly to 1.0 at *max_wheel_speed* (0.30 m/s at 100 %). Example: 0.10 m/s → d = 0.333 → duty = 0.25 + 0.75·0.333 = 0.50 → PWM ≈ 128. **Tuning:** if the motors hum without turning, raise min_duty; if the robot is too slow or too fast compared with the requested speed, correct max_wheel_speed with calibrate.py. Note that the safety layer also clamps at 0.20 m/s and the explorer asks only 0.12 m/s by default (SPEED in robot.env).

Integration (dead-reckoning)

Listing 4 – Pose update

```
def integrate(x, y, th, v, w, dt):
    mid = th + 0.5 * w * dt                 # use the mid-point heading: more accurate in turns
    return (x + v * math.cos(mid) * dt,
            y + v * math.sin(mid) * dt,
            normalize_angle(th + w * dt))
```

6.2 Talking to the Mega (motors.py)

motors.py defines a small interface (*set(left, right)*, *stop()*, *ticks()*) and three back-ends so the rest of the code never changes:

Class	Use
SerialMotors	The real robot: Pi → USB → Mega. Auto-reconnects, background reader thread, keep-alive.
GpioMotors	H-bridge wired straight to the Pi GPIO (gpiozero + lgpio). Not used in your setup.
DryRunMotors	Does nothing. Lets you test SLAM, odometry, Nav2 and the panel with no hardware (backend:=dry_run).

Key behaviours of **SerialMotors**:

- **set()** converts duty (−1…1) to integers (−255…255), applies *invert_left/right*, and writes "M l r". If the value is unchanged it only re-sends every 0.1 s (keep-alive) so the Mega watchdog never fires but the USB link is not flooded.
- **Reader thread** parses "T …" (encoders), "# …" (log) and "? …" (error) lines. If a read raises an exception, *_drop()* closes the port and the thread retries opening it every second: unplug and replug the cable and the robot reconnects by itself.
- **connected** is exposed so base_node can publish it in /base_status and health.py can raise "Mega link lost".
- **close()** sends S three times before closing: the robot stops when the node is stopped with Ctrl+C.

6.3 The base node (base_node.py)

This node is the bridge between ROS velocity commands and hardware. Its loop runs at 50 Hz (*loop_hz*):

Listing 5 – Core loop of base_node (condensed)

```
def tick(self):
    dt = (now - self.last_tick) ...                      # skip if dt <= 0 or > 0.5 s
    if (now - self.last_cmd) > cmd_timeout: cmd_v = cmd_w = 0      # command watchdog 0.5 s
    self.cur_v = k.slew(self.cur_v, self.cmd_v, max_accel * dt)     # acceleration limit
    self.cur_w = k.slew(self.cur_w, self.cmd_w, max_ang_accel * dt)
    vl, vr = k.wheel_speeds(self.cur_v, self.cur_w, self.sep)       # body -> sides
    dl = k.speed_to_duty(vl, self.max_speed, self.min_duty)         # sides -> duty
    dr = k.speed_to_duty(vr, self.max_speed, self.min_duty)
    self.motors.set(dl, dr)                                         # -> "M l r"
    if self.use_enc: self.update_from_encoders()
    else:            ...k.body_velocity(...); k.integrate(...)      # open-loop odometry
    self.publish(now.to_msg())                                      # /odom + TF odom->base_link
```

- **Slew-rate limiting** (max_accel 0.6 m/s², max_ang_accel 4 rad/s²) protects gears and the power supply from instant full-power reversals and reduces wheel slip (which would corrupt odometry).
- **Open-loop odometry** integrates what was *commanded* (after acceleration limiting). It cannot see slip. *linear_scale* and *angular_scale* correct systematic errors; skid-steer usually needs angular_scale < 1 because the tracks scrub when turning.
- **Encoder odometry** (use_encoders:=true) converts tick increments to distance with *ticks_per_meter_left/right*, then integrates distance and heading change. The covariance given to ROS is smaller (0.01/0.05 instead of 0.05/0.2) to tell SLAM that this odometry is more trustworthy.
- **Covariance:** the diagonal entries (x, y, yaw) tell SLAM how much to trust the motion model; z, roll and pitch are given a huge value (1000) because a planar robot does not measure them.

- **TF:** the node broadcasts *odom* → *base_link*. In the whole system it is the only publisher of this transform.

7. Perception: lidar driver, TF and scan filter

7.1 The YDLIDAR driver

The package *ydlidar_ros2_driver* wraps the YDLidar SDK (class CYdLidar). The driver node is itself the publisher of */scan*: it opens the serial port, starts the motor (DTR), reads packets, converts them to a LaserScan and publishes. Parameters live in robot_lidar/config/ydlidar_x2.yaml (baud 115200, single channel, range limits). The SDK needed CMake < 4 (or -DCMAKE_POLICY_VERSION_MINIMUM=3.5), which is why setup_pi.sh uses a private CMake 3 environment.

robot_lidar/launch/lidar.launch.py starts the driver and a **static transform** base_link → lidar_link built from lidar_x, lidar_z and **lidar_yaw**. If the cable side of the lidar points to the rear, use lidar_yaw:=3.14159 (LIDAR_YAW in robot.env). A wrong yaw makes the map rotate when the robot turns, and the safety layer would watch the wrong side, so check this first when the map looks strange.

7.2 scan_filter.py

Raw X2 scans contain invalid beams (0, NaN, out of range) and isolated "speckle" points. Ghost points create phantom obstacles and thicken walls in the map. The filter:

Listing 6 – Despeckle rule

```
valid[i] = (r == r) and not isinf(r) and r_min <= r <= r_max
tol = max(tol_abs, tol_rel * r_i)                    # 0.06 m or 4 % of the distance
keep beam i if at least `min_neighbors` beams within ±win (0.04 rad) differ by <= tol
else out[i] = inf
```

The node publishes the same LaserScan geometry (same angle_min, increment, number of beams) so downstream nodes need no changes: they simply read /scan_filtered instead of /scan. The cost is O(n·window) per scan, negligible on a Pi 5 for ~500 beams.

8. Command arbitration and safety: cmd_mux

mux.py is pure Python (no ROS) and holds the rules; **cmd_mux.py** is the ROS wrapper. This separation is what made it possible to test the safety logic with plain pytest.

8.1 Modes

Mode	Source listened to	Typical use
`stop`	`none – output is always zero`	Boot state of the service, after an e-stop
`manual`	`/cmd_vel_teleop`	You drive with the web panel, keyboard or a gamepad
`explore`	`/cmd_vel_auto`	Reactive autonomous wandering (builds a map by itself)
`nav`	`/cmd_vel`	Nav2 drives to a goal clicked on the map

Mode changes come from */robot_mode*. A mode switch is refused while the e-stop is latched. Releasing the e-stop leaves the mode on *stop*: you must choose a mode explicitly to move again. This avoids a robot that suddenly resumes the moment the button is released.

8.2 The step() function, line by line

Listing 7 – Decision order of the safety layer (condensed)

```
def step(self, t):
    info = dict(mode=..., estop=..., front=self.front, rear=self.rear, reason='ok',
                blind=(self.scan_t is None or t - self.scan_t > scan_timeout))
    if self.estopped:  return 0.0, 0.0, info(reason='estop')
    if self.mode == 'stop': return 0.0, 0.0, info(reason='stop')
    cmd = self.cmds.get(self.mode)
    if cmd is None or t - cmd[2] > source_timeout:        # dead-man: stale command
        return 0.0, 0.0, info(reason='no_command')
    v = clamp(cmd[0], -max_linear,  max_linear)
    w = clamp(cmd[1], -max_angular, max_angular)
    if (v != 0 or w != 0) and blind and block_when_blind: # no lidar -> no motion
        return 0.0, 0.0, info(reason='blind')
    if not blind:
        s = speed_scale(front if v > 0 else rear, stop_distance, slow_distance) if v != 0 else 1.0
        v *= s                                            # slow down / stop near obstacle
    return v, w, info
```

- The order matters: e-stop beats everything, then mode, then freshness, then clamps, then sensor-based limits.
- Obstacle scaling only multiplies the **linear** speed: rotating in place is still possible near a wall, which lets the robot turn away.
- When reversing (v < 0) the **rear** clearance is used; only the direction of motion is checked.

8.3 Clearance from the lidar scan

Listing 8 – corridor_clearance() and speed_scale()

```
for each beam i:
    x, y = r*cos(a), r*sin(a)            # a = angle_min + lidar_yaw + i*increment
    if |y| > half_width + margin: skip   # outside the corridor swept by the body
    if x > 0: fronts.append(x - front_offset)
    elif x < 0: rears.append(-x - rear_offset)
clearance = sorted(values)[hits-1]       # the 2nd nearest point: one noisy point cannot stop the robot

speed_scale(d) = 0                          if d <= stop_distance (0.10)
               = 1                          if d >= slow_distance (0.45)
               = (d - 0.10) / (0.45 - 0.10) otherwise (linear ramp)
```

Example: an obstacle 0.50 m ahead of the lidar is 0.50 − 0.205 = 0.295 m from the bumper, so the speed is scaled by (0.295 − 0.10)/0.35 = 0.56. At 0.305 m from the lidar it is 0.10 m from the bumper and the robot stops.

8.4 The ROS wrapper (cmd_mux.py)

- Subscribes to the three command topics with *lambda* callbacks that tag the source ("manual", "explore", "nav"), plus /robot_mode, /estop and the scan.
- A 20 Hz timer calls mux.step() and **always** publishes a Twist (zeros when blocked): base_node therefore sees a continuous stream and stays in sync.
- A 5 Hz timer publishes /robot_state JSON (mode, estop, front, rear, reason, v, w). Infinite clearances are sent as null because JSON has no infinity.
- On shutdown it publishes one zero Twist before exiting.

9. Autonomous exploration: explore.py

The explorer is a **reactive** behaviour: it needs no map. Pure logic lives in class *Wanderer* with a single method *step(ranges, angle_min, increment, r_min, r_max, now) → (v, w)*; ExploreNode feeds it each scan at 10 Hz and publishes on /cmd_vel_auto (remapped from /cmd_vel by the launch file).

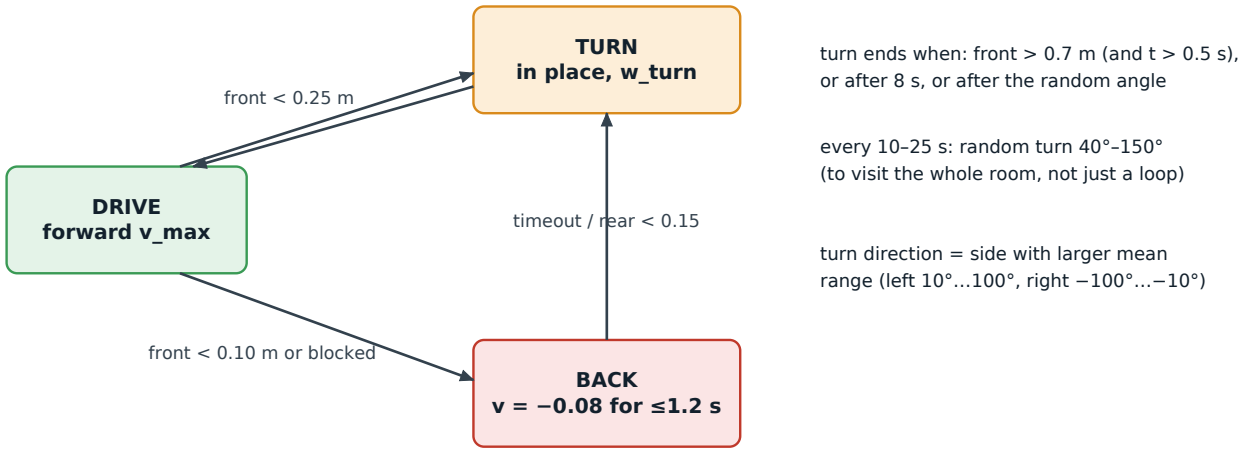


Figure 5 – Explorer state machine.

- **analyse()** computes in one pass: front/rear clearance inside the body corridor (HALF_W + 0.06), left/right side clearance, a *blocked* flag when any point is inside the circle of radius PIVOT_R = 0.19 m around the turning centre (so a turn in place would hit something), and the average range on the left and right to choose the open side.
- **start_delay** (10 s): the robot waits after the first scan so that SLAM can initialise and you can step back.
- **No scan = no motion**: if the scan is older than 1 s, a zero command is published.
- The explorer is intentionally simple. It does not guarantee full coverage: use *NAV=true* and click goals for deliberate coverage, or add frontier-based exploration later (see chapter 18).

10. Mapping and navigation configuration

10.1 slam_toolbox (config/slam_v2.yaml)

slam_toolbox runs in *online asynchronous* mapping mode: it processes scans as fast as it can without blocking, which suits a Pi. It is a **lifecycle node**: launching it directly leaves it unconfigured, which produced the empty-map problem earlier. bringup.launch.py therefore includes slam_toolbox's own online_async_launch.py with autostart:=true.

Parameter	Value	Why this value
`scan_topic`	/scan_filtered	Cleaned scans produce crisper walls
`resolution`	0.05 m	Good detail for rooms; larger maps cost more memory
`max_laser_range`	7.5 m	The X2 is noisy near its 8 m limit
`minimum_travel_distance / heading`	0.08 m / 0.08 rad	Process scans often so scan matching corrects drifting odometry
`ceres_loss_function`	HuberLoss	Robust to outliers (glass, shiny surfaces)
`do_loop_closing`	true	When the robot returns to a known place the whole map is corrected
`correlation_search_space_dimension`	0.6	Wide search because open-loop odometry may be off by several degrees
`distance/angle_variance_penalty`	0.6 / 0.8	Trust the scan matcher more than odometry
`map_update_interval`	1.5 s	Limits CPU load on the Pi

How to improve the map further (in order of impact): add wheel encoders (use_encoders), add an IMU for yaw, drive slowly (0.10–0.12 m/s) and smoothly, close loops deliberately, avoid long featureless corridors, and keep the lidar rigid and level.

10.2 Nav2 (optional, NAV=true)

nav2_params.py merges Nav2's default parameter file with config/nav2_overrides.yaml into a temporary file, then Nav2's *navigation_launch.py* starts planner, controller, behaviours and bt_navigator. No AMCL and no saved map are needed because slam_toolbox publishes the live /map and map → odom. Nav2 outputs /cmd_vel, which reaches the motors only in mode *nav* and always through cmd_mux. This part is the least tested: expect to tune footprint, inflation radius and speeds in nav2_overrides.yaml.

10.3 Health and autosave

- **health.py**: tracks the time of the last message of /scan, /odom, /map and the /base_status flags; evaluate() returns level ok / warn / error with a list of problems (lidar silent > 2 s, odometry silent, map not updating for 15 s, Mega link lost, Pi > 80 °C, free memory < 150 MB). Publishes /robot_health at 1 Hz; the panel shows it.
- **map_autosave.py**: every 180 s it saves the latest map with *map_saver_cli* to ~/maps/autosave/map_<time>.pgm/.yaml, keeps the last 10 and maintains latest.*. A power cut loses at most 3 minutes of mapping.

11. The PC web panel

The panel is an **HTTP server plus a ROS node in the same Python process**, run inside the Jazzy Docker container by pc/panel.sh. The browser never speaks ROS: it only fetches JSON and posts commands.

File	Role
`panel.py`	PanelNode: subscriptions (map, scan, state, health, TF), publishers (cmd_vel_teleop, robot_mode, estop, goal_pose), main() starts the HTTP server in a thread and rclpy.spin in the main thread.
`panel_core.py`	ROS-free: PanelState (thread-safe snapshot) and the ThreadingHTTPServer with the routes below. Testable without ROS.
`index.html`	The user interface: canvas map, lidar points, trail, drive pad and keys, mode buttons, e-stop (Esc), goal click-and-drag, map download.

Route	Method	Function
`/`	`GET`	The page, with a random token inserted
`/api/state?v=N`	`GET`	Snapshot: map (only if its version > N), pose, scan points, state, health
`/api/cmd`	`POST`	{v, w} → /cmd_vel_teleop
`/api/mode`	`POST`	{mode} → /robot_mode (validated against the 4 modes)
`/api/estop`	`POST`	{on} → /estop
`/api/goal`	`POST`	{x, y, yaw} → /goal_pose

11.1 Security model

- The server listens on **127.0.0.1 only**: other machines of the network cannot reach it.
- Each POST must carry the header *X-Panel-Token* equal to a random token that is embedded in the page when it is served.
- The **Host header** must be localhost:<port> or 127.0.0.1:<port>. Together with the token, this stops a malicious web page open in your browser (CSRF or DNS rebinding) from driving the robot.
- At most 4096 bytes of body are read per request.

11.2 How the map reaches the browser

on_map() converts the OccupancyGrid int8 data to bytes (−1 becomes 255), base64-encodes them and stores them with width, height, resolution and origin. The page requests /api/state?v=*last_version*; the map is included only when its version changed, which saves bandwidth. In the browser, each cell is painted on an off-screen canvas (unknown grey, free white, occupied black) and drawn with the transform world → pixel. Lidar points are already projected into the map frame by update_scan() using lookup_transform(map → lidar_link).

Note. The address-already-in-use error you saw means another process (or an old container) already uses port 8080. Run **PANEL_PORT=8099 ./pc/panel.sh** or stop the old container with **sudo docker ps** then **sudo docker stop <id>**.

12. Launch system and configuration files

Launch files in ROS 2 are Python programs that describe which nodes start, with which parameters and remappings.

Launch file	Starts
`robot_lidar/lidar.launch.py`	Lidar driver (model chosen by lidar:= x2/x4/g4), static TF, optional scan_check
`robot_base/robot.launch.py`	Includes lidar.launch.py + base_node with base.yaml, backend, serial_port and cmd_vel_topic
`robot_base/bringup.launch.py`	**The one you use.** robot.launch + scan_filter + cmd_mux + explore + health + map_autosave + slam_toolbox (+ Nav2 if nav:=true, + RViz if rviz:=true)
`robot_base/mapping.launch.py`	Development: SLAM only with the older parameter files
`robot_description/sim_lite.launch.py`	Simulation without Gazebo (fake_robot.py): used by the offline tests

12.1 The important remappings

Listing 9 – Wiring by names

```
explore:   /cmd_vel -> /cmd_vel_auto        /scan -> /scan_filtered
base_node: cmd_vel_topic = /cmd_vel_safe     (parameter, not a remap)
Nav2:      publishes /cmd_vel unchanged      (read by cmd_mux in mode 'nav')
panel:     publishes /cmd_vel_teleop         (read by cmd_mux in mode 'manual')
```

Remapping is the ROS way of rewiring a graph without editing code: the explorer was written to publish /cmd_vel, and the launch file silently redirects it. That is also why *explore.py* contains no knowledge of the mux.

12.2 Where to change what

I want to…	Edit	Parameter
Make the motors faster / slower	config/base.yaml	max_wheel_speed, min_duty, max_accel
Reverse a wheel direction	config/base.yaml	invert_left / invert_right
Change the real top speed	config/robot.yaml + base.yaml	cmd_mux.max_linear and base_node.max_linear
Make safety more or less cautious	config/robot.yaml	stop_distance, slow_distance, source_timeout
Fix the footprint / lidar position	config/robot.yaml, launch args	front_offset, rear_offset, half_width, lidar_x, lidar_yaw
Explorer speed	robot.env	SPEED (→ explore_speed)
Ports	robot.env	LIDAR_PORT, MEGA_PORT
Use encoders	base.yaml + firmware	use_encoders:=true, ENCODERS 1, ticks_per_meter_*
Map quality	config/slam_v2.yaml	resolution, minimum_travel_*, loop closing

13. Networking: Pi ↔ PC, Docker, DDS

- ROS 2 discovery uses multicast on the local network. Direct RJ45 cable: give both ends fixed addresses on the same subnet (Pi 192.168.50.2, PC e.g. 192.168.50.1).
- Both sides must export the same **ROS_DOMAIN_ID** (42, in robot.env and in pc/_common.sh).
- Docker is started with **--net=host** so the container uses the host network stack: DDS multicast works without port mapping, which is also why the panel binds port 8080 directly on the host.
- If discovery fails, set ROS_STATIC_PEERS to the Pi's IP (robot.env) or limit ROS_AUTOMATIC_DISCOVERY_RANGE; check with *ros2 topic list* inside the container.
- Do not mix distributions across the network: Pi Jazzy ↔ container Jazzy works; Pi Jazzy ↔ host Lyrical showed topics but no data (RViz empty).

14. Operation, calibration and tuning

14.1 Daily procedure

Where	Command	Result
Pi (SSH)	`./run_robot.sh`	Starts everything in mode manual; robot does not move by itself
PC	`./pc/panel.sh then open http://localhost:8080`	Live map, lidar, controls
PC	`./pc/mode.sh explore | nav | manual | stop`	Change mode without the browser
PC	`./pc/estop.sh`	Latched emergency stop
Pi	`NAV=true ./run_robot.sh`	Adds Nav2 (click goal in mode nav)
Pi	`./check_system.sh`	Layer-by-layer diagnosis (USB, nodes, topics, SLAM)
Pi	`./src/robot_base/scripts/save_map.sh name`	Save the map and the pose graph

14.2 Calibration sequence

- **Directions:** in manual mode press ↑: both sides must go forward. Otherwise set invert_left/invert_right.
- **Lidar yaw:** place an object in front of the robot; in the panel its points must appear in front. If they appear behind, set LIDAR_YAW=3.14159.

- **Straight line:** *calibrate.py straight* drives a known time and asks the real distance; it prints the suggested max_wheel_speed.
- **Rotation:** *calibrate.py turn* and measure the angle: it prints the suggested angular_scale.
- **Slow motors:** raise min_duty until both sides start reliably at the lowest speed; then raise SPEED gradually.
- **Map test:** drive a closed loop slowly and check that the walls close without a double line.

15. How to rebuild the project yourself

The best way to own the project is to rebuild it in layers, testing each one before adding the next. Each milestone below is independent and has a clear pass criterion.

#	Milestone	What you write / do	Pass criterion
1	Mega + H-bridge	Write the sketch of chapter 5; test with arduino_test.py or the serial monitor ("M 100 100").	Both sides turn forward and backward; stop after 0.4 s of silence
2	Serial from the Pi	Python script with pyserial: open port, wait 2 s, send "M l r" every 0.1 s.	Robot moves from a Python loop
3	Kinematics	kinematics.py (equations 1, speed_to_duty, integrate) and unit tests.	Tests pass; speed 0.1 → PWM ≈ 128
4	base_node	Subscribe /cmd_vel, publish /odom + TF (chapter 6.3), backend dry_run first.	ros2 topic echo /odom changes when you publish /cmd_vel
5	Lidar	Build the SDK and driver, set lidar yaml, add static TF.	ros2 topic hz /scan ≈ 10 Hz; points visible in RViz
6	SLAM	Launch slam_toolbox online_async with autostart; drive by hand.	/map is published and a room appears
7	Safety mux	Write mux.py first (tests!), then cmd_mux.py (chapter 8).	Robot stops when key is released, and before a wall
8	Explorer	Wanderer logic + node (chapter 9).	Robot wanders without hitting walls
9	PC panel	panel_core.py (HTTP + state) then panel.py then HTML.	Map and control work in the browser
10	Bring-up + service	One launch file, run_robot.sh, robot.service.	Power on → ready in mode stop

15.1 Minimal skeleton: a ROS 2 node in Python

Listing 10 – The pattern used by every node of the project

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class Hello(Node):
    def __init__(self):
        super().__init__('hello')
        self.declare_parameter('speed', 0.1)                    # configurable from YAML
        self.pub = self.create_publisher(Twist, '/cmd_vel_teleop', 10)
        self.create_timer(0.1, self.tick)                       # 10 Hz loop
    def tick(self):
        m = Twist(); m.linear.x = float(self.get_parameter('speed').value)
        self.pub.publish(m)

def main():
    rclpy.init(); node = Hello()
    try: rclpy.spin(node)
    except KeyboardInterrupt: pass
    finally: node.destroy_node(); rclpy.try_shutdown()
```

Every node in robot_base follows the same pattern: declare parameters, create publishers/subscribers/timers in *__init__*, keep decisions in separate pure functions, and register the executable in **setup.py** under entry_points (console_scripts) so that *ros2 run robot_base cmd_mux* and the launch file can find it. After editing, rebuild with *colcon build --packages-select robot_base*

and re-source install/setup.bash.

15.2 Package layout

Listing 11 – Workspace tree (abridged)

```
robot_ws/
├── README.md  robot.env  run_robot.sh  setup_pi.sh  check_system.sh  install_service.sh  run_all_tests.sh
├── src/
│   ├── robot_base/      base_node, cmd_mux, mux, explore, scan_filter, health, map_autosave,
│   │   │                kinematics, motors, nav2_params;  launch/  config/  scripts/  test/
│   ├── robot_lidar/     lidar.launch.py, ydlidar_*.yaml, scan_check.py
│   ├── ydlidar_ros2_driver/   C++ driver (needs the YDLidar SDK)
│   └── robot_description/     URDF, simulation, fake_robot.py, wander.py
├── firmware/mega_motor_driver/mega_motor_driver.ino   firmware/tools/arduino_test.py
└── pc/  panel.sh rviz.sh mode.sh estop.sh teleop_keyboard.sh  web_panel/(panel.py panel_core.py index.html)
```

16. Testing strategy

Test	What it proves	Status
test_mux_filter.py	Mux rules (e-stop, dead-man, blind, obstacle, clamps), clearance geometry, despeckle filter	Passed offline
test_explore_logic.py	Wanderer decisions	Passed offline
test_sim_integration.py	Teleop into walls and explorer through the mux in a simulated room: 0 collisions	Passed offline
test_serial_e2e.py, test_base_node_smoke.py	Serial protocol with a fake Mega, base_node behaviour	Passed offline
pc/web_panel/test_panel.py	Panel in real Chromium: rendering, buttons, keys, goal click, token and Host checks	Passed offline
firmware/host_test	Mega parsing logic compiled on a PC	Passed offline
Real launch of bringup on the Pi	All nodes + drivers together	**Not run**
Nav2 mode, Docker panel on your PC, service at boot	Integration on hardware	**Not run**
Calibration, motor tuning, SLAM quality on your floor	Performance	**To do on the robot**

Run all offline tests with **./run_all_tests.sh**. When something fails on the robot, use **./check_system.sh** to find the failing layer: USB device → driver → /scan → filter → TF → SLAM → /map → mux → base_node → serial → Mega.

17. Troubleshooting map

Symptom	Likely cause	What to do
Panel: Address already in use	Port 8080 taken (old container, other app)	PANEL_PORT=8099 ./pc/panel.sh or stop the old container
Panel opens but no map	PC and Pi not on the same domain / distro mismatch / SLAM not active	ros2 topic list in the container; check ROS_DOMAIN_ID; ros2 lifecycle get /slam_toolbox
Lidar points but no map	slam_toolbox unconfigured	Use bringup (it autostarts it); look for "Registering sensor" in the log
Map rotates when the robot turns	Wrong lidar_yaw or large angular drift	Check LIDAR_YAW; calibrate angular_scale; drive slower
Robot does not move in manual	Mode is stop, e-stop latched, no lidar (blind), or source timeout	Read /robot_state "reason" in the panel; choose Manual
Motors hum, do not turn	min_duty too low or weak supply	Raise min_duty; check battery
One side does not turn	Wiring or one H-bridge channel	Test pins with arduino_test.py
Mega not found	ttyACM number changed	Use /dev/serial/by-id/… in robot.env
USB over-current in dmesg	Pi power supply or short	5 V / 5 A supply; no motor power from the Pi
ros2: command not found	ROS not sourced	source /opt/ros/jazzy/setup.bash and install/setup.bash

18. Limitations and roadmap

18.1 What this system is not

- **Not a certified industrial product.** The e-stop is software, there is no safety-rated controller, no bumpers or cliff sensors, and the lidar sees only one horizontal plane (not steps, cables or low objects).
- Open-loop odometry limits map quality in large empty spaces and long corridors.
- Exploration is reactive, not frontier-based; Nav2 is configured but not field-tuned.

18.2 Recommended upgrades, in order

Upgrade	Benefit	Effort
Physical e-stop switch on the motor battery	Real safety cut-off	Low
Wheel encoders (set ENCODERS 1, use_encoders)	Much better odometry and maps; closed-loop speed control	Medium
IMU (MPU-6050/BNO055) + robot_localization EKF	Reliable yaw	Medium
Front bumper / ultrasonic or ToF ring	Detect what the lidar cannot see	Medium
Closed-loop PID speed controller on the Mega	Constant speed under load / battery drop	Medium
Frontier exploration (explore_lite) instead of the wanderer	Complete map coverage	Medium
Battery monitoring (voltage divider on an analog pin → /battery)	Know when to stop	Low
Watchdog and diagnostics (diagnostic_updater), log recording (rosbag2)	Fleet-grade supportability	Medium

Appendix A – Command cheat sheet

Commands

```
# ----- Pi (SSH) -----
ssh <user>@192.168.50.2
cd ~/robot_ws && ./setup_pi.sh              # once (or after code changes)
./run_robot.sh                              # manual mode, nothing moves alone
NAV=true ./run_robot.sh                     # with Nav2
MODE=explore ./run_robot.sh                 # start exploring (use with care)
./check_system.sh                           # diagnosis
sudo ./install_service.sh                   # start at boot in mode stop

# ----- PC -----
cd robot_ws && ./pc/panel.sh                # http://localhost:8080   (PANEL_PORT=8099 if busy)
./pc/mode.sh manual|explore|nav|stop
./pc/estop.sh
./pc/rviz.sh                                # optional RViz in the Jazzy container

# ----- useful ROS 2 commands -----
ros2 node list ; ros2 topic list ; ros2 topic hz /scan ; ros2 topic echo /robot_state --once
ros2 run tf2_ros tf2_echo map base_link
ros2 lifecycle get /slam_toolbox
ros2 param get /base_node min_duty ; ros2 param set /base_node min_duty 0.35
```

Appendix B – Glossary of the code names

Name	Meaning
`base_link`	Frame at the robot body centre (turning centre)
`lidar_link`	Frame at the lidar; offset from base_link by lidar_x/z/yaw
`odom`	Frame of dead-reckoning (smooth but drifting)
`map`	Fixed frame of the SLAM map (globally consistent)
`duty`	PWM fraction −1…1 (Pi side); PWM value −255…255 on the Mega
`slew`	Limit on how fast a value may change per second (acceleration limit)
`dead-man`	Motion only while commands keep arriving; silence stops the robot
`blind`	No recent lidar scan; the mux refuses to move the robot
`clearance`	Free distance between body edge and nearest obstacle in the driving corridor
`mux`	Multiplexer: selects one of several inputs according to a mode